InternNova – Week 1 Internship Assignment

Python Fundamentals for Data Analytics

Student Name: Adarsh (edit if needed)

College Name: Add your college name

Branch: Add your branch

Duration: 1 Week (7 Days) **Maximum Marks:** 100

Submission Deadline: 12 November 2026

How to use this report

This report contains the source code, a brief explanation, and an illustrative output for each task. Run each Python file on your computer and capture your own screenshots. Replace sample/personal details where indicated before submitting.

Task-wise marks

Task	Topic	Marks
1	Python Basics	10
2	Variables & Data Types	10
3	Operators	10
4	Conditional Statements	10
5	Loops	10
6	Functions	10
7	Strings & Collections	15
8	File Handling	10
9	Mini Project	15
	Total	100

Task 1: Python Basics (10 Marks)

Explanation: This program displays a welcome message, accepts the student's name, college, and branch using input(), and prints the information in a formatted layout.

Illustrative output: Example: Name = Adarsh; College = ABC College; Branch = Computer Science. The program prints all three details under STUDENT INFORMATION.

Python source code:

```
# Task 1: Python Basics
print("Welcome to the Python Fundamentals for Data Analytics Assignment!")

name = input("Enter your name: ")
college = input("Enter your college name: ")
branch = input("Enter your branch: ")

print("\n========== STUDENT INFORMATION ==========")
print(f"Name     : {name}")
print(f"College : {college}")
print(f"Branch  : {branch}")
print("=========================================")
```

Task 2: Variables & Data Types (10 Marks)

Explanation: The program creates an integer, float, string, and Boolean variable. The type() function displays the data type of each value.

Illustrative output: Example output: age = 20 | type = ; percentage = 85.5 | type = ; student_name = Adarsh | type = ; is_student = True | type = .

Python source code:

```
# Task 2: Variables and Data Types
age = 20                 # Integer
percentage = 85.5        # Float
student_name = "Adarsh"  # String
is_student = True        # Boolean

print("Variable values and their data types:")
print("age =", age, "| type =", type(age))
print("percentage =", percentage, "| type =", type(percentage))
print("student_name =", student_name, "| type =", type(student_name))
print("is_student =", is_student, "| type =", type(is_student))
```

Task 3: Operators – Calculator (10 Marks)

Explanation: The calculator accepts two numbers and performs addition, subtraction, multiplication, division, and modulus. It checks for zero before division and modulus.

Illustrative output: For inputs 10 and 3: Addition = 13; Subtraction = 7; Multiplication = 30; Division = 3.333...; Modulus = 1.

Python source code:

```
# Task 3: Operators - Simple Calculator
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print("\n========== CALCULATOR ==========")
print(f"Addition    :  {num1  +  num2}")
print(f"Subtraction  :  {num1  -  num2}")
print(f"Multiplication : {num1 * num2}")

if num2 != 0:
    print(f"Division        : {num1 / num2}")
    print(f"Modulus         : {num1 % num2}")
else:
    print("Division        : Not possible (division by zero)")
    print("Modulus         : Not possible (division by zero)")
print("================================")
```

Task 4: Conditional Statements – Grade Calculator (10 Marks)

Explanation: The program uses if, elif, and else to assign a grade based on marks. It also checks that marks are between 0 and 100.

Illustrative output: Example: input 82 produces Grade: B. 90 or more gives A; 75–89.99 gives B; 60–74.99 gives C; below 60 gives Fail.

Python source code:

```
# Task 4: Conditional Statements - Grade Calculator
marks = float(input("Enter your marks (0-100): "))

if marks < 0 or marks > 100:
    print("Invalid marks. Please enter a value from 0 to 100.")
elif marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
elif marks >= 60:
    print("Grade: C")
else:
    print("Grade: Fail")
```

Task 5: Loops (10 Marks)

Explanation: A for loop prints numbers 1–20 and generates a multiplication table. A while loop prints even numbers from 2 to 50.

Illustrative output: The output includes 1 through 20, the entered number multiplied by 1 through 10, and even numbers 2, 4, 6, ... 50.

Python source code:

```
# Task 5: Loops

# 1. Print numbers from 1 to 20 using a for loop
print("Numbers from 1 to 20:")
for number in range(1, 21):
    print(number, end=" ")
print()

# 2. Multiplication table of a number
table_number = int(input("\nEnter a number for its multiplication table: "))
print(f"Multiplication table of {table_number}:")
for i in range(1, 11):
    print(f"{table_number} x {i} = {table_number * i}")

# 3. Print even numbers from 1 to 50 using a while loop
print("\nEven numbers from 1 to 50:")
number = 2
while number <= 50:
    print(number, end=" ")
    number += 2
print()
```

Task 6: Functions (10 Marks)

Explanation: Two user-defined functions are created: square() returns the square of a number, and average() returns the average of three numbers. Both are called using user input.

Illustrative output: Example: square of 5 = 25. Average of 10, 20, and 30 = 20.00.

Python source code:

```
# Task 6: User-Defined Functions

def square(number):
    """Return the square of a number."""
    return number ** 2

def average(first, second, third):
    """Return the average of three numbers."""
    return (first + second + third) / 3

value = float(input("Enter a number to find its square: "))
print(f"Square of {value} = {square(value)}")

a = float(input("\nEnter the first number: "))
b = float(input("Enter the second number: "))
c = float(input("Enter the third number: "))
print(f"Average = {average(a, b, c):.2f}")
```

Task 7: Strings & Collections (15 Marks)

Explanation: This program demonstrates string methods upper(), lower(), replace(), and find(); list methods append(), remove(), and sort(); tuple creation and indexing; a dictionary; and set add()/remove() methods.

Illustrative output: The output shows the transformed string, updated sorted list, selected tuple items, student dictionary values, and updated set. Set display order may vary.

Python source code:

```
# Task 7: Strings and Collections

# 1. String operations
message = "Python is useful for data analytics"
print("Original string:", message)
print("Uppercase:", message.upper())
print("Lowercase:", message.lower())
print("Replace Python with SQL:", message.replace("Python", "SQL"))
print("Position of 'useful':", message.find("useful"))

# 2. List operations
marks = [78, 92, 65, 85]
print("\nOriginal list:", marks)
marks.append(88)
print("After append(88):", marks)
marks.remove(65)
print("After remove(65):", marks)
marks.sort()
print("After sort():", marks)

# 3. Tuple creation and indexing
student = ("Riya", 21, "Computer Science")
print("\nTuple:", student)
print("First item:", student[0])
print("Third item:", student[2])

# 4. Dictionary storing student information
student_info = {
    "name": "Riya",
    "roll_no": 101,
    "branch": "Computer Science",
    "marks": 88
}
print("\nStudent dictionary:", student_info)
print("Student name:", student_info["name"])

# 5. Set operations
subjects = {"Python", "SQL", "Excel"}
print("\nOriginal set:", subjects)
subjects.add("Power BI")
print("After add():", subjects)
subjects.remove("Excel")
print("After remove():", subjects)
```

Task 8: Basic File Handling (10 Marks)

Explanation: The program accepts a student's introduction, writes it to my_introduction.txt using write mode, then opens the file in read mode and displays its contents.

Illustrative output: Example: after entering the student's details, the same introduction is displayed from the text file.

Python source code:

```
# Task 8: Basic File Handling
name = input("Enter your name: ")
college = input("Enter your college name: ")
branch = input("Enter your branch: ")

introduction = (
    f"My name is {name}. I study at {college}. "
    f"My branch is {branch}. I am learning Python for data analytics."
)

file_name = "my_introduction.txt"

# Create a file and write the introduction
with open(file_name, "w", encoding="utf-8") as file:
    file.write(introduction)

print(f"\nIntroduction successfully written to '{file_name}'.")

# Open the same file and display its contents
with open(file_name, "r", encoding="utf-8") as file:
    content = file.read()

print("\nContents of the file:")
print(content)
```

Task 9: Mini Project – Student Record Management System (15 Marks)

Explanation: The mini project uses a list of dictionaries to store student records. A menu lets the user add, display, search by name, delete by roll number, or exit. Duplicate roll numbers and empty fields are checked.

Illustrative output: Example workflow: choose 1 to add a student; choose 2 to display records; choose 3 to search by name; choose 4 to delete by roll number; choose 5 to exit.

Python source code:

```
# Task 9: Student Record Management System
# Records are stored in a list of dictionaries during program execution.

students = []

def add_student():
    name = input("Enter student name: ").strip()
    roll_no = input("Enter roll number: ").strip()
    branch = input("Enter branch: ").strip()

    if not name or not roll_no or not branch:
        print("All fields are required.")
        return

    if any(student["roll_no"].lower() == roll_no.lower() for student in students):
        print("A student with this roll number already exists.")
        return

    students.append({
        "name": name,
        "roll_no": roll_no,
        "branch": branch
    })
    print("Student record added successfully.")

def display_students():
    if not students:
        print("No student records found.")
        return

    print("\n========== STUDENT RECORDS ==========")
    for index, student in enumerate(students, start=1):
        print(f"{index}. Name: {student['name']} | "
              f"Roll No: {student['roll_no']} | Branch: {student['branch']}")
    print("=====================================")

def search_student():
    search_name = input("Enter the name to search: ").strip().lower()
    matches = [student for student in students
               if search_name in student["name"].lower()]

    if matches:
        print("\nMatching student record(s):")
        for student in matches:
            print(f"Name: {student['name']}, Roll No: {student['roll_no']}, "
                  f"Branch: {student['branch']}")
    else:
        print("Student not found.")

def delete_student():
    roll_no = input("Enter the roll number of the student to delete: ").strip()
    for student in students:
        if student["roll_no"].lower() == roll_no.lower():
            students.remove(student)
            print("Student record deleted successfully.")
            return
    print("No student found with that roll number.")

def main():
    while True:
        print("\n===== STUDENT RECORD MANAGEMENT SYSTEM =====")
        print("1. Add student")
        print("2. Display all students")
        print("3. Search student by name")
        print("4. Delete student by roll number")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ").strip()

        if choice == "1":
            add_student()
```

```
        elif choice == "2":
            display_students()
        elif choice == "3":
            search_student()
        elif choice == "4":
            delete_student()
        elif choice == "5":
            print("Thank you for using the Student Record Management System.")
            break
        else:
            print("Invalid choice. Please enter a number from 1 to 5.")

if __name__== "__main__":
    main()
```